Подготовка к тестированию «Большие вызовы»

Направление: Большие данные, искусственный интеллект, автоматизированные системы и информационная безопасность

Часть 1

Региональный центр “Альтаир”

Автор: Моисеенко Елизавета Сергеевна

Содержание

1 Математика **5**

1.1 Многочлены и неравенства 6

1.1.1 Теория 6

1.1.2 Разбор задач прошлых лет 8

1.1.3 Задачи для самостоятельного решения 12

1.2 Арифметика остатков и прогрессии 17

1.2.1 Теория 17

1.2.2 Разбор задач прошлых лет 21

1.2.3 Задачи для самостоятельного решения 30

1.3 Комбинаторика 33

1.3.1 Теория 33

1.3.2 Задачи для самостоятельного решения 44

2 Программирование **47**

2.1 Базовые конструкции Python 47
2.1.1 Разбор задач прошлых лет 47
2.2 Работа со строками 50
2.2.1 Разбор задач прошлых лет 50
2.2.2 Практические шаблоны 56
2.3 Словари и частотный анализ 57
2.3.1 Разбор задач прошлых лет 57
2.3.2 Практические шаблоны 63
2.4 Работа с файлами 64
2.4.1 Разбор задач прошлых лет 64
2.4.2 Практические шаблоны 69
2.5 Графы 70
2.5.1 Разбор задач прошлых лет 70
2.5.2 Практические шаблоны 75
2.6 Алгоритмы обработки последовательностей 77
2.6.1 Разбор задач прошлых лет 77
2.6.2 Практические шаблоны 82
2.7 Алгоритмические конструкции 83
2.7.1 Разбор задач прошлых лет 83
2.7.2 Практические шаблоны 88

Желаем удачи в подготовке к тестированию! Данная методичка поможет вам систематизировать знания, разобрать типовые задачи и отработать навыки решения.

В части 1 данной методички представлена необходимая теория, решение задач прошлых лет и задания для самостоятельной отработки по следующим темам: математика, программирование.

В части 2 представлены темы по направлениям машинное обучение, анализ данных и робототехника.

Формат проведения

В рамках первого тура заключительного этапа конкурса участникам в зависимости от выбранного профиля будут предложены задачи по темам математики, программирования, анализа данных, искусственного интеллекта и робототехники.

В профиле “Анализ данных и искусственный интеллект” будут представлены задачи по темам математики, анализа данных и искусственного интеллекта.

В профиле “Программная инженерия и безопасность” — задачи по темам математики и программирования.

В профиле “Робототехника” — задачи по темам математики и робототехники.

Тестирование проводится в очном формате за общественными компьютерами, на которых может не быть заранее установленного привычного программного обеспечения для программирования.

Поэтому рекомендуется заранее определить онлайн-компилятор, который можно использовать для решения задач.

Далее по каждому блоку задач будут перечислены основные темы, примеры решений заданий прошлых лет, необходимая теория, а также задания для самостоятельного решения.

Если у вас возникнут вопросы или трудности в подготовке, вы можете обратиться по почте: `mes@donso.su`

Разбалловка за 2025 год

Направление	Сумма баллов
Математика	5.5
Анализ данных	4.0
Машинное обучение и ИИ	4.5
Программирование	8.5
Робототехника	8.5

Итого баллов: 14.

1 Математика

Особенности задач по математике

При подготовке к задачам по математике особое внимание следует уделить дополнительным главам, выходящим за пределы стандартной школьной программы.

Алгебра и теория чисел

- многочлены
- неравенства
- арифметика остатков
- сравнения по модулю

Логические задачи

- перечислительная комбинаторика
- графы
- логика
- рассуждения от противного

1.1 Многочлены и неравенства

1.1.1 Теория

Определение

Многочлен — это выражение вида

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

где

- $a_0, a_1, \ldots, a_n$ — коэффициенты многочлена,
- n — степень многочлена,
- $a_n \neq 0$.

Степенью многочлена называется наибольшая степень переменной x.

—

Формулы Виета Рассмотрим квадратное уравнение

$$x^2 + px + q = 0.$$

Пусть x_1 и x_2 — его корни.

Формула

Формулы Виета:

$$x_1 + x_2 = -p$$

$$x_1 x_2 = q$$

Обозначим

$$S = x_1 + x_2, \qquad P = x_1 x_2.$$

Тогда можно выразить другие симметрические выражения через S и P.

—

Формула

Сумма квадратов корней

$$x_1^2 + x_2^2 = (x_1 + x_2)^2 - 2x_1x_2$$

$$x_1^2 + x_2^2 = S^2 - 2P$$

—

Формула

Сумма кубов корней

$$x_1^3 + x_2^3 = (x_1 + x_2)^3 - 3x_1x_2(x_1 + x_2)$$

$$x_1^3 + x_2^3 = S^3 - 3PS$$

—

Формула

Сумма обратных корней

$$\frac{1}{x_1} + \frac{1}{x_2} = \frac{x_1 + x_2}{x_1x_2} = \frac{S}{P}$$

—

Сумма коэффициентов многочлена Пусть задан многочлен

$$P(x) = a_nx^n + a_{n-1}x^{n-1} + \cdots + a_1x + a_0.$$

Тогда сумма всех коэффициентов равна значению многочлена при $x = 1$:

$$P(1) = a_n + a_{n-1} + \cdots + a_1 + a_0.$$

Чтобы найти сумму коэффициентов многочлена, достаточно подставить в него значение $x = 1$.

1.1.2 Разбор задач прошлых лет

2024

Задача

Квадратный трёхчлен

$$x^2 - 12x + 7$$

имеет корни x_1, x_2. Найдите

$$\frac{1}{x_1} + \frac{1}{x_2}.$$

Ответ введите в виде дроби.

Решение

Используем формулу

$$\frac{1}{x_1} + \frac{1}{x_2} = \frac{x_1 + x_2}{x_1 x_2}.$$

По формулам Виета для уравнения $x^2 - 12x + 7 = 0$:

$$x_1 + x_2 = 12, \quad x_1 x_2 = 7.$$

Тогда

$$\frac{1}{x_1} + \frac{1}{x_2} = \frac{12}{7}.$$

Задача

Пусть x_1, x_2 — корни уравнения

$$x^2 - 12x + 7 = 0.$$

Найдите

$$x_1^2 + x_2^2.$$

Решение

Используем формулу

$$x_1^2 + x_2^2 = (x_1 + x_2)^2 - 2x_1x_2.$$

По Виету

$$x_1 + x_2 = 12, \quad x_1x_2 = 7.$$

Тогда

$$x_1^2 + x_2^2 = 12^2 - 2 \cdot 7 = 144 - 14 = 130.$$

Задача

Пусть x_1, x_2 — корни уравнения

$$x^2 - 12x + 7 = 0.$$

Найдите

$$x_1^3 + x_2^3.$$

Решение

Используем формулу

$$x_1^3 + x_2^3 = (x_1 + x_2)^3 - 3x_1x_2(x_1 + x_2).$$

По Виету

$$S = x_1 + x_2 = 12, \quad P = x_1x_2 = 7.$$

Тогда

$$x_1^3 + x_2^3 = S^3 - 3PS = 12^3 - 3 \cdot 7 \cdot 12 = 1728 - 252 = 1476.$$

2025

Задача

При каком значении a сумма корней уравнения

$$3x^2 - 9ax + a = 0$$

равна 9?

Решение

По формулам Виета

$$x_1 + x_2 = -\frac{-9a}{3} = 3a.$$

По условию

$$3a = 9 \Rightarrow a = 3.$$

Задача

Чему равна сумма коэффициентов многочлена

$$(x^{2025} - x^{2024} + 1)^{2026}?$$

Решение

Сумма коэффициентов равна значению многочлена при $x = 1$.

$$P(x) = (x^{2025} - x^{2024} + 1)^{2026}$$

$$P(1) = (1 - 1 + 1)^{2026} = 1.$$

Ответ: 1.

Задача

Найдите сумму корней уравнения

$$(\sqrt{x})^{2024} + (\sqrt{1-x})^{2024} = 1.$$

Решение

Перепишем уравнение

$$x^{1012} + (1-x)^{1012} = 1.$$

Так как выражения под корнем неотрицательны,

$$0 \le x \le 1.$$

Проверим края:

$$x = 0: \quad 0^{1012} + 1^{1012} = 1$$

$$x = 1: \quad 1^{1012} + 0^{1012} = 1$$

Обе точки подходят.
Если $0 < x < 1$, то $x^{1012} < 1$ и $(1-x)^{1012} < 1$, причём при большой степени обе величины очень малы, поэтому их сумма меньше 1.
Следовательно, других решений нет.

$$x_1 = 0, \quad x_2 = 1$$

Сумма корней

$$x_1 + x_2 = 1.$$

1.1.3 Задачи для самостоятельного решения

1. Найдите сумму корней уравнения
$$x^2 - 7x + 5 = 0.$$

2. Найдите произведение корней уравнения
$$x^2 - 4x + 3 = 0.$$

3. Уравнение
$$x^2 - 6x + 8 = 0$$
имеет корни x_1, x_2. Найдите
$$x_1 + x_2.$$

4. Найдите сумму коэффициентов многочлена

$$(x^2 + x + 1)^5.$$

5. Найдите сумму коэффициентов многочлена

$$(x^3 - x + 2)^6.$$

6. Уравнение

$$x^2 - 5x + 6 = 0$$

имеет корни x_1, x_2. Найдите

$$\frac{1}{x_1} + \frac{1}{x_2}.$$

7. Для того же уравнения найдите

$$x_1^2 + x_2^2.$$

8. Уравнение

$$x^2 - 4x + 2 = 0$$

имеет корни x_1, x_2. Найдите

$$x_1^3 + x_2^3.$$

9. Уравнение

$$x^2 - 3x + 1 = 0$$

имеет корни x_1, x_2. Найдите

$$x_1^2 + x_2^2.$$

10. Если

$$x_1 + x_2 = 6,$$

$$x_1 x_2 = 8,$$

найдите

$$x_1^2 + x_2^2.$$

11. Если

$$x_1 + x_2 = 5,$$

$$x_1 x_2 = 6,$$

найдите

$$x_1^3 + x_2^3.$$

12. Если

$$x_1 + x_2 = 4,$$

$$x_1 x_2 = 3,$$

найдите

$$x_1^4 + x_2^4.$$

13. Уравнение

$$x^2 - 2x + 1 = 0$$

имеет корни x_1, x_2. Найдите

$$x_1^5 + x_2^5.$$

14. Если

$$x_1 + x_2 = 7,$$

$$x_1 x_2 = 10,$$

найдите

$$x_1^3 + x_2^3.$$

15. При каком значении a сумма корней уравнения

$$2x^2 - ax + 3 = 0$$

равна 5?

16. При каком a уравнение

$$x^2 - ax + 9 = 0$$

имеет сумму корней 8?

17. При каком a уравнение

$$3x^2 - 6ax + 1 = 0$$

имеет сумму корней 4?

18. Найдите сумму коэффициентов многочлена

$$(x^{100} - x^{99} + 1)^{50}.$$

19. Найдите сумму коэффициентов многочлена

$$(x^{2025} - x^{2024} + 1)^{2026}.$$

20. Найдите сумму коэффициентов многочлена

$$(x^5 - x^4 + x^3 + 1)^{10}.$$

21. Решите уравнение

$$(\sqrt{x})^4 + (\sqrt{1-x})^4 = 1.$$

22. Решите уравнение

$$(\sqrt{x})^6 + (\sqrt{1-x})^6 = 1.$$

23. Решите уравнение

$$(\sqrt{x})^{10} + (\sqrt{1-x})^{10} = 1.$$

24. Если

$$x_1 + x_2 = 3,$$

$$x_1 x_2 = 1,$$

найдите

$$x_1^6 + x_2^6.$$

25. Уравнение

$$x^2 - 3x + 1 = 0$$

имеет корни x_1, x_2. Найдите

$$x_1^6 + x_2^6.$$

1.2 Арифметика остатков и прогрессии

1.2.1 Теория

Определение

Если число a делится на число m с остатком r, то

$$a = mq + r,$$

где

$$q \text{ — целое число}, \qquad 0 \leq r < m.$$

Пример

$$17 = 5 \cdot 3 + 2$$

Следовательно, остаток от деления 17 на 5 равен 2.

—

Определение

Числа a и b называются **сравнимыми по модулю** m, если их разность делится на m.
Это записывается так:

$$a \equiv b \pmod{m}$$

и означает, что

$$a - b$$

делится на m.

Пример

$$17 \equiv 2 \pmod 5$$

поскольку

$$17 - 2 = 15$$

делится на 5.

—

Правило

Если

$$a \equiv b \pmod m \qquad c \equiv d \pmod m$$

то выполняются следующие свойства:

Сложение

$$a + c \equiv b + d \pmod m$$

Вычитание

$$a - c \equiv b - d \pmod m$$

Умножение

$$ac \equiv bd \pmod m$$

Возведение в степень

$$a^k \equiv b^k \pmod m$$

—

Последняя цифра любого числа — это остаток от деления на 10.

$$a \equiv b \pmod{10}$$

Пример

$$12347 \equiv 7 \pmod{10}$$

Следовательно, последняя цифра числа равна 7.

—

Правило

Степени многих чисел образуют **циклы последних цифр**.
Степени числа 2

$$2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 8, \quad 2^4 = 16$$

Последние цифры:

$$2, 4, 8, 6$$

Цикл длины 4.
Степени числа 3

$$3, 9, 7, 1$$

Цикл длины 4.
Степени числа 7

$$7, 9, 3, 1$$

Цикл длины 4.
Степени числа 9

$$9, 1$$

Цикл длины 2.

—

Правило

Сумма первых n нечётных чисел равна

$$1 + 3 + 5 + \cdots + (2n - 1) = n^2.$$

Пример

$$1 + 3 + 5 + 7 + 9 = 25.$$

—

Определение

Если числа идут с одинаковым шагом

$$a,\ a + d,\ a + 2d, \ldots$$

то такая последовательность называется **арифметической прогрессией**.

Правило

Сумма первых n членов арифметической прогрессии:

$$S = \frac{n(a_1 + a_n)}{2}.$$

Количество членов прогрессии:

$$n = \frac{a_n - a_1}{d} + 1.$$

—

Правило

Для натуральных чисел m и n

$$\gcd(a^m - 1,\ a^n - 1) = a^{\gcd(m,n)} - 1.$$

Пример

$$\gcd(10^{27}-1,\ 10^{18}-1)$$

$$\gcd(27,18)=9$$

$$=10^9-1$$

—

Правило

Телескопическая сумма имеет вид

$$\frac{1}{n(n+1)}.$$

Разложение:

$$\frac{1}{n(n+1)}=\frac{1}{n}-\frac{1}{n+1}.$$

Пример

$$\frac{1}{20\cdot 21}+\frac{1}{21\cdot 22}$$

$$=\left(\frac{1}{20}-\frac{1}{21}\right)+\left(\frac{1}{21}-\frac{1}{22}\right)$$

1.2.2 Разбор задач прошлых лет

2023

Задача

Найдите все натуральные $n \leq 300$, для которых

$$n^2 - 16n + 63$$

делится на 143.

Решение

Разложим выражение:

$$n^2 - 16n + 63 = (n-7)(n-9)$$

Разложим число:

$$143 = 11 \cdot 13$$

Нужно

$$(n-7)(n-9) \equiv 0 \pmod{143}$$

то есть

$$n \equiv 7 \quad \text{или} \quad n \equiv 9$$

Решения имеют вид

$$n = 7 + 143k, \qquad n = 9 + 143k$$

При $n \leq 300$ получаем

$$7,\ 9,\ 150,\ 152,\ 293,\ 295.$$

—

Задача

На плоскости нарисован правильный 30-угольник. Сколько различных правильных пятиугольников можно составить из его вершин?

Решение

Правильный k-угольник внутри правильного n-угольника получается, если выбирать вершины через одинаковый шаг.
Для пятиугольника

$$\frac{30}{5} = 6$$

шаг между вершинами равен 6.
Начать можно с любой из 30 вершин.
Но каждый пятиугольник будет посчитан 5 раз (по количеству его вершин).
Следовательно

$$\frac{30}{5} = 6$$

Ответ: 6.

—

Задача

Найдите последнюю цифру числа

$$1 + 3 + 5 + 7 + \cdots + 2013.$$

Решение

Это сумма первых нечётных чисел.
Последнее число:

$$2n - 1 = 2013$$

$$n = 1007$$

Сумма первых n нечётных чисел:

$$n^2$$

Следовательно

$$1007^2$$

Последняя цифра определяется по

$$7^2 = 49$$

Ответ: 9.

—

Задача

Найдите последнюю цифру произведения

$$1 \cdot 3 \cdot 5 \cdot 7 \cdots 2013.$$

Решение

В произведении присутствует множитель 5.
Все остальные множители нечётные.
Поэтому произведение заканчивается на 5.

—

Задача

Найдите сумму всех пятизначных натуральных чисел.

Решение

Пятизначные числа:

$$10000 \ldots 99999$$

Количество чисел:

$$99999 - 10000 + 1 = 90000$$

Это арифметическая прогрессия.

$$S = \frac{n(a_1 + a_n)}{2}$$

$$S = \frac{90000(10000 + 99999)}{2}$$

$$= 45000 \cdot 109999$$

$$= 4\,949\,955\,000$$

—

Задача

Найдите

$$\gcd(10^{27} - 1,\ 10^{18} - 1).$$

Ответ представьте в десятичной записи.

Решение

Используем формулу

$$\gcd(a^m - 1,\ a^n - 1) = a^{\gcd(m,n)} - 1$$

$$\gcd(27, 18) = 9$$

Следовательно

$$10^9 - 1$$

$$= 999\,999\,999$$

—

Задача

Найдите

$$\gcd(10^{27} + 1,\ 10^{18} + 1).$$

Решение

Если степени нечётные, то

$$\gcd(a^m + 1,\ a^n + 1) = a^{\gcd(m,n)} + 1$$

$$\gcd(27, 18) = 9$$

Следовательно

$$10^9 + 1$$

$$= 1\,000\,000\,001$$

—

Задача

Одной кастрюли супа хватает на 2 взрослых или 5 детей.
Есть 5 кастрюль супа и 15 детей.
Сколько взрослых можно накормить оставшимся супом?

Решение

15 детей требуют

$$15/5 = 3$$

кастрюли супа.
Остаётся

$$5 - 3 = 2$$

кастрюли.
Каждая кормит 2 взрослых.

$$2 \cdot 2 = 4$$

Ответ: 4.

—

Задача

На доске написано число 2023.
За один ход можно заменить число x на

$$x - 3$$

или

$$2x + 6.$$

Какие числа из промежутка $11 \ldots 19$ можно получить?

Решение

Заметим:

$$x - 3 \equiv x \pmod 3$$

$$2x + 6 \equiv 2x \equiv x \pmod 3$$

Обе операции сохраняют остаток по модулю 3.

$$2023 \equiv 1 \pmod 3$$

Среди чисел $11 \ldots 19$ остаток 1 имеют

$$13,\ 16,\ 19.$$

Ответ: $13, 16, 19$.

—

2025

Задача

Найдите последнюю цифру числа

$$1^2 + 2^2 + \cdots + 2025^2.$$

Решение

Используем формулу

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Берём $n = 2025$.
Рассмотрим по модулю 10:

$$2025 \equiv 5$$

$$2026 \equiv 6$$

$$4051 \equiv 1$$

Получаем

$$\frac{5 \cdot 6 \cdot 1}{6} = 5$$

Ответ: 5.

—

Задача

Вычислите

$$2025 \left(\frac{1}{20 \cdot 21} + \frac{1}{21 \cdot 22} + \cdots + \frac{1}{2024 \cdot 2025} \right).$$

Решение

Используем разложение

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1}.$$

Сумма телескопируется:

$$\frac{1}{20} - \frac{1}{2025}.$$

Умножаем:

$$2025\left(\frac{1}{20} - \frac{1}{2025}\right)$$

$$= \frac{2025}{20} - 1$$

$$= 101.25 - 1$$

$$= 100.25.$$

1.2.3 Задачи для самостоятельного решения

1. Найдите остаток от деления

$$23 \mod 5$$

2. Найдите остаток от деления

$$57 \mod 9$$

3. Найдите последнюю цифру числа

$$17^2$$

4. Найдите последнюю цифру числа

$$23^3$$

5. Найдите последнюю цифру числа

$$7^4$$

6. Найдите последнюю цифру числа

$$3^{20}$$

7. Найдите последнюю цифру числа

$$2^{25}$$

8. Найдите последнюю цифру числа

$$9^{17}$$

9. Найдите последнюю цифру числа

$$1 + 3 + 5 + \cdots + 199$$

10. Найдите сумму

$$1 + 2 + 3 + \cdots + 100$$

11. Найдите сумму

$$1 + 3 + 5 + \cdots + 999$$

12. Найдите сумму всех четырёхзначных чисел.

13. Найдите сумму

$$5 + 10 + 15 + \cdots + 500$$

14. Найдите последнюю цифру числа

$$1^2 + 2^2 + \cdots + 100^2$$

15. Найдите

$$\gcd(10^{12} - 1,\ 10^8 - 1)$$

16. Найдите

$$\gcd(2^{15} - 1,\ 2^9 - 1)$$

17. Найдите
$$\gcd(3^{20}-1,\ 3^{12}-1)$$

18. Найдите сумму
$$\frac{1}{1\cdot 2}+\frac{1}{2\cdot 3}+\cdots+\frac{1}{10\cdot 11}$$

19. Найдите
$$100\left(\frac{1}{1\cdot 2}+\cdots+\frac{1}{99\cdot 100}\right)$$

20. Найдите последнюю цифру числа
$$1^2+2^2+\cdots+1000^2$$

21. Найдите последнюю цифру числа
$$7^{100}$$

22. Найдите последнюю цифру числа
$$8^{123}$$

23. Найдите
$$\gcd(10^{50}-1,\ 10^{20}-1)$$

24. Найдите сумму
$$\frac{1}{10\cdot 11}+\cdots+\frac{1}{100\cdot 101}$$

25. Найдите последнюю цифру числа
$$1^2+2^2+\cdots+2025^2$$

1.3 Комбинаторика

1.3.1 Теория

Определение

Комбинаторика — раздел математики, который изучает способы подсчёта количества различных вариантов.

—

Правило

Правило суммы
Если объект можно выбрать одним из нескольких способов, то общее количество вариантов равно сумме вариантов.

Пример

Можно выбрать:

- 3 красных шара
- 5 синих шаров

Всего способов выбрать шар:

$$3 + 5 = 8$$

—

Правило

Правило произведения
Если действие состоит из нескольких шагов, то общее число вариантов равно произведению числа вариантов на каждом шаге.

Пример

Есть:

- 4 рубашки
- 3 пары брюк

Количество вариантов одежды:

$$4 \cdot 3 = 12$$

—

Определение

Факториал
Произведение всех натуральных чисел от 1 до n:

$$n! = 1 \cdot 2 \cdot 3 \cdots n$$

Пример

$$3! = 6$$

$$4! = 24$$

$$5! = 120$$

—

Определение

Перестановки
Перестановки — это все возможные способы упорядочить n различных объектов.

Правило

Если есть n различных элементов, то число перестановок равно

$$n!$$

Пример

Сколько способов расставить буквы A, B, C?

$$3! = 6$$

$$ABC,\ ACB,\ BAC,\ BCA,\ CAB,\ CBA$$

—

Правило

Если некоторые элементы одинаковые, то число перестановок:

$$\frac{n!}{k_1!k_2!\ldots}$$

где $k_1, k_2, \ldots$ — количества одинаковых элементов.

Пример

Слово **ПАПА**
4 буквы, из них

- П — 2
- А — 2

$$\frac{4!}{2!2!} = 6$$

—

Определение

Сочетания
Сочетания — выбор элементов **без учёта порядка**.

Правило

Формула сочетаний:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Пример

Выбрать 2 учеников из 5:

$$C_5^2 = 10$$

—

Определение

Размещения
Размещения — выбор элементов **с учётом порядка**.

Правило

$$A_n^k = \frac{n!}{(n-k)!}$$

Пример

Сколько кодов длины 3 можно составить из 5 цифр?

$$A_5^3 = \frac{5!}{2!} = 60$$

—

Определение

Распределение объектов
Если нужно распределить n одинаковых объектов между k людьми.

Правило

Число решений уравнения

$$x_1 + x_2 + \cdots + x_k = n$$

равно

$$C_{n+k-1}^{k-1}$$

Пример

10 яблок распределяются между 3 детьми.

$$x_1 + x_2 + x_3 = 10$$

Количество способов:

$$C_{12}^{2}$$

—

Определение

Вероятность
Вероятность показывает, насколько вероятно событие.

Правило

$$P = \frac{\text{благоприятные исходы}}{\text{все возможные исходы}}$$

Пример

Игральный кубик.
Всего исходов:

$$6$$

Вероятность выпадения числа 3:

$$\frac{1}{6}$$

—

Правило

Если события независимы:

$$P(A \cap B) = P(A) \cdot P(B)$$

Пример

Два броска кубика.
Вероятность получить 3 и 4:

$$\frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$$

—

Если бросают два кубика, то всего возможных исходов:

$$6 \cdot 6 = 36$$

Пример

Сумма 7.
Подходящие пары:

$$(1,6),(2,5),(3,4),(4,3),(5,2),(6,1)$$

Всего:

$$6$$

Вероятность:

$$\frac{6}{36}=\frac{1}{6}$$

—

Правило

Число строк длины n из символов 0 и 1:

$$2^n$$

Пример

Строки длины 5:

$$2^5=32$$

Задача

Сколько различных слов можно получить, переставляя буквы в слове **ВЫЗОВЫ**, если слово не может начинаться с буквы Ы?

Решение

Повторы букв:

$$^{-}2, \qquad ^{-}2$$

Всего перестановок:

$$\frac{6!}{2!2!} = \frac{720}{4} = 180$$

Теперь посчитаем слова, которые **начинаются на Ы**.
Фиксируем первую букву Ы.
Остаётся 5 букв:

$$, , , ,$$

Перестановок:

$$\frac{5!}{2!} = 60$$

Подходящих слов:

$$180 - 60 = 120$$

—

Задача

Два кубика бросаются одновременно.
Какова вероятность того, что сумма выпавших чисел делится на 4?

Решение

Всего исходов:

$$6 \cdot 6 = 36$$

Суммы, делящиеся на 4:

$$4, 8, 12$$

Посчитаем способы.

сумма	способов
4	3
8	5
12	1

Всего:

$$3 + 5 + 1 = 9$$

Вероятность:

$$\frac{9}{36} = \frac{1}{4}$$

—

Задача

Из натуральных чисел от 1 до 500 случайно выбирается одно. Какова вероятность того, что оно делится на 3 и не делится на 12?

Решение

Числа, кратные 3:

$$\left\lfloor \frac{500}{3} \right\rfloor = 166$$

Числа, кратные 12:

$$\left\lfloor \frac{500}{12} \right\rfloor = 41$$

Подходящие:

$$166 - 41 = 125$$

Вероятность:

$$\frac{125}{500} = \frac{1}{4}$$

—

Задача

Даша нашла 20 бусин:

$$5 \text{ красных}, \quad 6 \text{ синих}, \quad 7 \text{ жёлтых}, \quad 2 \text{ зелёных}.$$

Браслет считается красивым, если любые две соседние бусины имеют разные цвета.
Какое максимальное количество бусин можно использовать?

Решение

Самый частый цвет — жёлтые:

$$7$$

Остальные бусины:

$$5 + 6 + 2 = 13$$

Чтобы можно было чередовать цвета, должно выполняться условие

$$\text{остальные} \geq \text{максимум} - 1$$

Проверяем:

$$13 \geq 7 - 1$$

Условие выполняется.
Следовательно, можно использовать все бусины.

$$5 + 6 + 7 + 2 = 20$$

—

Задача

Сколько существует строк длины 10, состоящих из цифр 0 и 1, если в них нет двух нулей подряд?

Решение

Обозначим количество таких строк через

$$a_n$$

Если строка заканчивается на 1, то перед ней может быть любая строка длины $n-1$.
Если строка заканчивается на 0, то предыдущий символ обязательно 1.
Получаем рекурсию:

$$a_n = a_{n-1} + a_{n-2}$$

Это последовательность Фибоначчи.

$$a_{10} = 144$$

1.3.2 Задачи для самостоятельного решения

1. В классе учится 9 мальчиков и 7 девочек. Сколькими способами можно выбрать одного ученика?

2. В классе учится 10 мальчиков и 8 девочек. Сколькими способами можно выбрать одного мальчика и одну девочку?

3. В классе учится 12 учеников. Сколькими способами можно выбрать команду из 2 человек?

4. В классе учится 8 мальчиков и 6 девочек. Сколькими способами можно выбрать команду из 2 мальчиков?

5. В классе учится 11 мальчиков и 9 девочек. Сколькими способами можно выбрать команду из 2 мальчиков и 1 девочки?

6. В классе учится 15 учеников. Сколькими способами можно выбрать команду из 3 человек?

7. В классе учится 10 мальчиков и 7 девочек. Сколькими способами можно выбрать команду из 4 человек, состоящую из 2 мальчиков и 2 девочек?

8. В классе учится 8 мальчиков и 6 девочек. Сколькими способами можно выбрать команду из 3 человек, если в ней должна быть хотя бы одна девочка?

9. В классе учится 6 мальчиков и 5 девочек. Сколькими способами можно выбрать команду из 4 человек, если в ней не более двух мальчиков?

10. Сколько различных слов можно получить, переставляя буквы слова **ТОМ**?

11. Сколько различных слов можно получить, переставляя буквы слова **ТЕСТ**?

12. Сколько различных слов можно получить, переставляя буквы слова **МОЛОКО**?

13. Сколько различных слов можно получить, переставляя буквы слова **ПРОГРАММА**?

14. При броске шестигранного кубика найдите вероятность того, что выпадет число больше 4.

    Ответ укажите в виде дроби.

15. Два раза бросают шестигранный кубик. Найдите вероятность того, что сумма выпавших чисел равна 8.

16. Два раза бросают шестигранный кубик. Найдите вероятность того, что сумма делится на 3.

17. В мешке лежат 5 красных и 7 синих шаров. Случайно вытаскивают один шар.

    Какова вероятность того, что он окажется красным?

18. В мешке лежат 6 красных, 4 синих и 5 зелёных шаров. Случайно вытаскивают один шар.

    Какова вероятность того, что он не зелёный?

19. Три ученика делят 7 одинаковых конфет. Сколькими способами можно распределить конфеты, если каждый может получить любое количество?

20. Четыре ученика делят 10 одинаковых яблок. Сколькими способами можно распределить яблоки, если некоторые могут не получить ни одного?

21. Сколько существует строк длины 5, состоящих только из цифр 0 и 1?

22. Сколько существует строк длины 6, состоящих только из цифр 0 и 1, если строка начинается с 1?

23. Сколько существует строк длины 7, состоящих из 0 и 1, если не допускается два нуля подряд?

24. В коробке лежат 5 красных и 3 синих шара. Случайно вытаскивают два шара без возвращения.

    Какова вероятность того, что оба шара окажутся красными?

    Ответ укажите в виде дроби.

25. Три раза бросают шестигранный кубик. Найдите вероятность того, что сумма выпавших чисел равна 9.

    Ответ укажите в виде дроби.

2 Программирование

2.1 Базовые конструкции Python

2.1.1 Разбор задач прошлых лет

2023

Задача

Дан фрагмент псевдокода.

```
d = {'a':1, 'b':2, 'c':3, 'd':4, '-':-1}
s = "ab--b-daa--a"

result = 0
prev_minus = False

for i in range(len(s)):
    letter = s[i]

    if letter == '-':
        if not prev_minus:
            result += d[letter]
        prev_minus = True
    else:
        result += d[letter]
        prev_minus = False
```

В качестве ответа укажите значение переменной `result` после выполнения программы.

Решение

Словарь задаёт следующие значения символов:

$$a \to 1, \quad b \to 2, \quad c \to 3, \quad d \to 4, \quad - \to -1.$$

Строка имеет вид

$$\texttt{ab-b-daa-a}.$$

Переменная `prev_minus` используется для того, чтобы учитывать только **первый символ '-'** в последовательности подряд идущих минусов.
Пройдём по строке:

Символ	Действие	Значение result
a	+1	1
b	+2	3
-	-1	2
-	пропускаем	2
b	+2	4
-	-1	3
d	+4	7
a	+1	8
a	+1	9
-	-1	8
-	пропускаем	8
a	+1	9

Итоговое значение переменной

$$\texttt{result} = 9.$$

2024

Задача

Ниже приведена функция на Python, которая должна находить максимальное значение в двумерном массиве.
Однако в коде содержатся ошибки.

```
def find_max(arr):
    found_max = float('inf')

    for i in range(len(arr)):
        for j in range(len(arr[-1])):
            if arr[j][i] > found_max:
                found_max = arr[i][j]
                break

    return found_max
```

Найдите все ошибки в реализации функции. Всего их три. В ответе укажите номера строк функции.

Решение

Рассмотрим код.
Ошибка 1.
В строке

```
found_max = float('inf')
```

переменная инициализируется бесконечно большим числом. Однако функция ищет **максимум**, поэтому начальное значение должно быть очень маленьким:

```
found_max = float('-inf')
```

Ошибка 2.
В условии используется обращение

```
arr[j][i]
```

Однако внешний цикл перебирает строки, а внутренний - столбцы. Поэтому правильное обращение к элементу матрицы:

```
arr[i][j]
```

Ошибка 3.
В коде присутствует оператор

```
break
```

который преждевременно завершает внутренний цикл. Из-за этого часть элементов матрицы может не проверяться.
Следовательно, оператор `break` необходимо удалить.
Ответ:

2, 6, 8

2.2 Работа со строками

2.2.1 Разбор задач прошлых лет

2024

Задача

В архиве находятся текстовые файлы с описанием химических элементов. Необходимо подсчитать суммарное количество букв, которые встречаются во всех файлах.
Буквой считается любой латинский символ из диапазона `a-z` или `A-Z`.
Для проверки того, является ли символ буквой, в Python можно использовать метод `isalpha()`.

Решение

Необходимо последовательно прочитать все символы файлов и проверить, является ли каждый символ буквой.
Для этого можно использовать метод строки `isalpha()`, который возвращает `True`, если символ является буквой.
Алгоритм решения:

1. открыть файл,
2. прочитать его содержимое,
3. пройти по всем символам строки,
4. увеличить счётчик, если символ является буквой.

Пример реализации на Python:

```
count = 0

with open("file.txt") as f:
    text = f.read()

    for char in text:
        if char.isalpha():
            count += 1

print(count)
```

Таким образом, программа подсчитывает количество букв во всём тексте.

2024

Задача

В архиве находятся текстовые файлы. Необходимо определить три слова, которые встречаются в файлах наиболее часто.
При подсчёте:

- не учитывается регистр символов;
- из слов необходимо удалить лишние символы (`.` `,` `:` `;` `(` `)`).

Слова в ответе необходимо привести к нижнему регистру и расположить в порядке убывания частоты.

Решение

Такая задача решается с помощью **частотного словаря**.
Алгоритм решения:

1. считать текст файла;
2. привести текст к нижнему регистру;
3. удалить лишние символы;
4. разбить текст на слова;
5. подсчитать частоту каждого слова.

Пример реализации:

```
import re

freq = {}

with open("file.txt") as f:
    text = f.read().lower()

    words = re.findall(r"[a-z]+", text)

    for word in words:
        freq[word] = freq.get(word, 0) + 1

top_words = sorted(freq.items(),
                   key=lambda x: x[1],
                   reverse=True)

print(top_words[:3])
```

В результате будут получены три слова, которые встречаются чаще всего.

2025

Задача

В файле задана бинарная строка s, состоящая только из символов `0` и `1`. Гарантируется, что количество нулей в строке равно количеству единиц.
Необходимо разбить строку на максимальное число подряд идущих подстрок так, чтобы в каждой подстроке количество нулей было равно количеству единиц.
Найдите максимальное количество таких подстрок.

Решение

Решение основано на использовании **баланса** между нулями и единицами.
Будем считать:

$$1 \to +1, \qquad 0 \to -1.$$

Если сумма этих значений равна нулю, то в текущем фрагменте строки одинаковое количество нулей и единиц.
Алгоритм:

1. пройти по строке слева направо;
2. поддерживать текущий баланс;
3. если баланс становится равным нулю, то найден новый корректный фрагмент.

Пример реализации:

```
balance = 0
count = 0

for c in s:
    if c == '1':
        balance += 1
    else:
        balance -= 1

    if balance == 0:
        count += 1

print(count)
```

Каждый раз, когда баланс становится равным нулю, мы можем завершить текущую подстроку и начать следующую.

2.2.2 Практические шаблоны

Пример

Перебор символов строки

```
s = "HelloWorld"

for c in s:
    print(c)
```

Пример

Проверка типа символа

```
s = "Hello 123"

letters = 0
digits = 0
spaces = 0

for c in s:
    if c.isalpha():
        letters += 1
    if c.isdigit():
        digits += 1
    if c.isspace():
        spaces += 1

print(letters, digits, spaces)
```

Пример

Приведение строки к нижнему регистру и разбиение на слова

```
s = "Hello World Python"

s = s.lower()
words = s.split()
```

```
for w in words:
    print(w)
```

Пример

Извлечение слов из текста

```
import re

text = "Hello, world! Python is great."

words = re.findall(r"[a-z]+", text.lower())

for w in words:
    print(w)
```

2.3 Словари и частотный анализ

2.3.1 Разбор задач прошлых лет

2023

Задача

В файле записано 100 000 целых неотрицательных чисел. Числа записаны в одну строку и разделены пробелами.
Необходимо определить число, которое встречается в файле **наиболее часто**.
Если таких чисел несколько, можно вывести любое из них.

Решение

Такая задача удобно решается с помощью **словаря**.
Словарь позволяет хранить пары

$$\text{ключ} \to \text{значение}.$$

В данной задаче:

$$\text{число} \to \text{количество появлений}.$$

Алгоритм решения:

1. считать все числа из файла;
2. для каждого числа увеличить его счётчик в словаре;
3. найти ключ с максимальным значением.

Пример реализации:

```
freq = {}

with open("numbers.txt") as f:
    numbers = map(int, f.read().split())

    for x in numbers:
        freq[x] = freq.get(x, 0) + 1

answer = max(freq, key=freq.get)

print(answer)
```

Функция `freq.get(x,0)` позволяет получить значение по ключу, а если ключ отсутствует — вернуть 0.

2024

Задача

В архиве находятся текстовые файлы. Необходимо определить слова, которые встречаются в тексте наиболее часто.
При подсчёте:

- не учитывается регистр символов;
- необходимо удалить лишние символы (`. , : ; ( )`);
- слова в ответе должны быть приведены к нижнему регистру.

Найдите три слова с наибольшей частотой.

Решение

Решение снова основано на использовании **частотного словаря**. Алгоритм:

1. считать текст файла;
2. привести текст к нижнему регистру;
3. извлечь слова из текста;
4. подсчитать частоту каждого слова;
5. отсортировать слова по частоте.

Пример реализации:

```
import re

freq = {}

with open("text.txt") as f:
    text = f.read().lower()

words = re.findall(r"[a-z]+", text)

for word in words:
    freq[word] = freq.get(word, 0) + 1

top_words = sorted(freq.items(),
                   key=lambda x: x[1],
                   reverse=True)

print(top_words[:3])
```

Функция `re.findall()` позволяет выделить из текста только последовательности букв.

2025

Задача

В файле хранится набор записей в формате JSON. Каждая строка файла содержит описание действия пользователя в социальной сети. Каждая запись содержит поля:

- `user_id` — идентификатор пользователя;
- `action` — тип действия;
- `date` — дата действия.

Возможные действия:

Действие	Баллы
post	3
comment	2
friendship	1

Необходимо определить пользователя, который набрал наибольшее количество баллов в **марте 2024 года**.

Решение

Для решения задачи необходимо:

1. считать каждую строку файла;
2. преобразовать строку JSON в словарь;
3. проверить дату события;
4. увеличить счётчик баллов пользователя.

Пример реализации:

```
import json

scores = {}

with open("events.txt") as f:

    for line in f:
        event = json.loads(line)

        if event["date"].startswith("2024-03"):

            user = event["user_id"]
            action = event["action"]

            if action == "post":
                score = 3
            elif action == "comment":
                score = 2
            else:
                score = 1

            scores[user] = scores.get(user, 0) +
               score

answer = max(scores, key=scores.get)

print(answer)
```

Таким образом мы подсчитываем суммарную активность каждого пользователя и выбираем максимальное значение.

2.3.2 Практические шаблоны

Пример

Частотный словарь

```
data = [3, 5, 3, 7, 5, 3]

freq = {}

for x in data:
    freq[x] = freq.get(x, 0) + 1

print(freq)
```

Пример

Поиск элемента с максимальной частотой

```
data = [3, 5, 3, 7, 5, 3]

freq = {}

for x in data:
    freq[x] = freq.get(x, 0) + 1

result = max(freq, key=freq.get)

print(result)
```

Пример

Сортировка словаря по частоте

```
data = ["a", "b", "a", "c", "b", "a"]

freq = {}

for x in data:
    freq[x] = freq.get(x, 0) + 1
```

```
result = sorted(freq.items(),
                key=lambda x: x[1],
                reverse=True)

print(result)
```

2.4 Работа с файлами

2.4.1 Разбор задач прошлых лет

2024

Задача

Дан Python-код функции, которая должна вычислять количество непустых строк в файле.
Строка считается непустой, если она содержит хотя бы один символ, отличный от пробельного.
В коде содержатся ошибки.

```
def not_null_lines(file_path):
    cnt = 0

    with open(file_path, "w") as f:
        for line in f:
            if line.isspace():
                pass

        cnt += 1

    return cnt
```

Найдите все ошибки в реализации функции. Всего их две.

Решение

Рассмотрим код.

Ошибка 1.

Файл открывается в режиме записи:

```
open(file_path, "w")
```

Однако функция должна читать файл. Поэтому необходимо использовать режим

```
open(file_path, "r")
```

Ошибка 2.

Счётчик строк увеличивается вне условия:

```
cnt += 1
```

Он должен увеличиваться только для строк, которые не состоят из пробелов.

Правильная логика:

```
if not line.isspace():
    cnt += 1
```

Таким образом, ошибки находятся в строках

4, 9.

2024

Задача

В архиве находятся текстовые файлы с описаниями химических элементов.

Необходимо определить суммарное количество букв, которые встречаются во всех файлах.

Буквой считается любой символ латинского алфавита.

Решение

Необходимо считать содержимое файла и проверить каждый символ.
Если символ является буквой, увеличиваем счётчик.
Для проверки можно использовать метод строки `isalpha()`.
Пример решения:

```
count = 0

with open("elements.txt") as f:
    text = f.read()

    for char in text:
        if char.isalpha():
            count += 1

print(count)
```

Метод `isalpha()` возвращает `True`, если символ является буквой.

2025

Задача

В файле записаны события пользователей социальной сети в формате JSON. Каждая строка файла содержит отдельную запись. Каждая запись содержит поля:

- `user_id` — идентификатор пользователя,
- `action` — действие пользователя,
- `date` — дата события.

Необходимо определить пользователя, который набрал наибольшее количество баллов в марте 2024 года.
Баллы начисляются следующим образом:

Действие	Баллы
post	3
comment	2
friendship	1

Решение

Файл содержит строки в формате JSON. Каждую строку необходимо преобразовать в словарь Python.
Для этого используется функция `json.loads()`.
Алгоритм решения:

1. прочитать файл построчно;
2. преобразовать строку JSON в словарь;
3. проверить дату события;
4. увеличить счётчик баллов пользователя.

Пример реализации:

```
import json

scores = {}

with open("events.txt") as f:

    for line in f:
        event = json.loads(line)

        if event["date"].startswith("2024-03"):

            user = event["user_id"]
            action = event["action"]

            if action == "post":
                score = 3
            elif action == "comment":
                score = 2
            else:
                score = 1

            scores[user] = scores.get(user, 0) +
               score

answer = max(scores, key=scores.get)

print(answer)
```


Таким образом мы подсчитываем суммарную активность каждого пользователя и выбираем максимальное значение.

2.4.2 Практические шаблоны

Пример

Загрузка файла в Google Colab

```
from google.colab import files

uploaded = files.upload()
```

Пример

Чтение всего файла

```
with open("file.txt") as f:
    text = f.read()

print(text)
```

Пример

Чтение файла построчно

```
with open("file.txt") as f:
    for line in f:
        print(line.strip())
```

Пример

Чтение чисел из файла

```
with open("numbers.txt") as f:
    numbers = list(map(int, f.read().split()))

print(numbers)
```

Пример

Чтение JSON-файла

```python
import json

with open("events.txt") as f:
    for line in f:
        event = json.loads(line)
        print(event)
```

2.5 Графы

2.5.1 Разбор задач прошлых лет

2023

Задача

В архиве содержится файл, в котором записана матрица смежности неориентированного графа.
Число строк в файле равно числу вершин графа. Если в i-й строке и j-м столбце стоит 1, то между вершинами i и j существует ребро. Если стоит 0, то ребра нет.
Необходимо определить длину кратчайшего пути между заданными парами вершин.

Решение

Для поиска кратчайшего пути в невзвешенном графе используется алгоритм **поиска в ширину (BFS)**.
Идея алгоритма:

1. начинаем поиск из стартовой вершины;
2. посещаем всех соседей;
3. затем соседей соседей;
4. продолжаем, пока не достигнем нужной вершины.

При этом расстояние до вершины увеличивается на 1 на каждом шаге.
Пример реализации:

```
from collections import deque

def bfs(graph, start):

    n = len(graph)
    dist = [-1] * n

    q = deque([start])
    dist[start] = 0

    while q:

        v = q.popleft()

        for u in range(n):

            if graph[v][u] == 1 and dist[u] == -1:

                dist[u] = dist[v] + 1
                q.append(u)

    return dist
```

Массив `dist` хранит расстояние от стартовой вершины до всех остальных вершин.

2024

Задача

Дан неориентированный граф, заданный матрицей смежности. Для каждой пары вершин необходимо определить, существует ли между ними путь.

Решение

Если между двумя вершинами существует путь, то они принадлежат одной **компоненте связности**.
Проверить наличие пути можно с помощью алгоритма поиска в ширину (BFS) или поиска в глубину (DFS).
Алгоритм:

1. запускаем поиск из первой вершины;
2. отмечаем все достижимые вершины;
3. если вторая вершина была достигнута, то путь существует.

Пример реализации:

```
def dfs(graph, v, visited):

    visited[v] = True

    for u in range(len(graph)):

        if graph[v][u] == 1 and not visited[u]:
            dfs(graph, u, visited)
```

После выполнения DFS можно проверить, была ли посещена нужная вершина.

2024

Задача

Дан граф, заданный матрицей смежности.
Необходимо определить количество компонент связности в графе.

Решение

Компонента связности — это множество вершин, между любой парой которых существует путь.
Алгоритм:

1. создаём массив посещённых вершин;
2. перебираем все вершины;
3. если вершина не посещена, запускаем из неё DFS;
4. увеличиваем число компонент.

Пример реализации:

```
def count_components(graph):

    n = len(graph)
    visited = [False] * n

    def dfs(v):
        visited[v] = True

        for u in range(n):
            if graph[v][u] == 1 and not visited[u]:
                dfs(u)

    components = 0

    for v in range(n):

        if not visited[v]:
            dfs(v)
            components += 1

    return components
```

2025

Задача

Дан неориентированный граф, заданный матрицей смежности.
Необходимо определить вершину с максимальной степенью.
Если таких вершин несколько, можно вывести любую.

Решение

Степень вершины — это количество рёбер, которые инцидентны данной вершине.
В матрице смежности степень вершины равна сумме элементов соответствующей строки.
Алгоритм:

1. для каждой вершины посчитать сумму строки;
2. выбрать вершину с максимальным значением.

Пример реализации:

```
max_degree = -1
vertex = -1

for i in range(len(graph)):

    degree = sum(graph[i])

    if degree > max_degree:
        max_degree = degree
        vertex = i

print(vertex)
```

Таким образом определяется вершина, имеющая наибольшее количество соседей.

2.5.2 Практические шаблоны

Пример

Степень вершины в матрице смежности

```
graph = [
[0,1,1],
[1,0,1],
[1,1,0]
]

for i in range(len(graph)):
    degree = sum(graph[i])
    print(i, degree)
```

Пример

Поиск в ширину (BFS)

```
from collections import deque

graph = {
0:[1,2],
1:[0,3],
2:[0],
3:[1]
}

start = 0

visited = set([start])
q = deque([start])

while q:

    v = q.popleft()

    for u in graph[v]:

        if u not in visited:
            visited.add(u)
            q.append(u)

print(visited)
```

2.6 Алгоритмы обработки последовательностей

2.6.1 Разбор задач прошлых лет

2023

Задача

Даны две функции.

```
def f(l1, l2):
    res = []
    i = 0
    j = 0

    while (i < len(l1)) and (j < len(l2)):

        if l1[i] < l2[j]:
            res.append(l1[i])
            i += 1
        else:
            res.append(l2[j])
            j += 1

    return res + l1[i:] + l2[j:]

def g(l1):

    l2 = []

    for x in l1:

        x2 = x * x

        if x < 0:
            l1.append(x2)
        else:
            l2.append(x2)

    return f(list(reversed(l1)), l2)
```

Функция g вызывается для списка

$$[-9, -7, -3, 0, 0, 2, 3, 10].$$

Определите результат работы программы.

Решение

Сначала вычисляются квадраты всех элементов списка.

$$\begin{aligned} -9 &\to 81 \\ -7 &\to 49 \\ -3 &\to 9 \\ 0 &\to 0 \\ 0 &\to 0 \\ 2 &\to 4 \\ 3 &\to 9 \\ 10 &\to 100 \end{aligned}$$

Квадраты отрицательных чисел попадают в первый список, а квадраты неотрицательных — во второй.
Получаем

$$l_1 = [81, 49, 9], \qquad l_2 = [0, 0, 4, 9, 100].$$

Первый список разворачивается:

$$[9, 49, 81].$$

Функция `f` выполняет **слияние двух отсортированных списков** (операция *merge* из алгоритма сортировки слиянием).
В результате получаем

$$[0, 0, 4, 9, 9, 49, 81, 100].$$

2025

Задача

Дана бинарная строка s, состоящая только из символов `0` и `1`. Гарантируется, что количество нулей равно количеству единиц.
Необходимо разбить строку на максимальное число подряд идущих подстрок так, чтобы в каждой подстроке количество нулей было равно количеству единиц.
Найдите максимальное число таких подстрок.

Решение

Будем отслеживать **баланс** между нулями и единицами.
Положим

$$1 \to +1, \qquad 0 \to -1.$$

Если сумма равна нулю, значит в текущем фрагменте одинаковое количество нулей и единиц.
Алгоритм:

1. пройти по строке;
2. поддерживать текущий баланс;
3. когда баланс становится равным нулю, найдена новая корректная подстрока.

Пример реализации:

```
balance = 0
count = 0

for c in s:

    if c == '1':
        balance += 1
    else:
        balance -= 1

    if balance == 0:
        count += 1

print(count)
```

Каждый раз, когда баланс становится равным нулю, мы можем завершить текущую подстроку.

2025

Задача

Дана последовательность чисел. Необходимо найти максимальную сумму подпоследовательности, в которой нельзя выбирать два соседних элемента.

Решение

Для решения задачи используется **динамическое программирование**.
Обозначим через $dp[i]$ максимальную сумму, которую можно получить, рассматривая первые i элементов последовательности.
Для каждого элемента есть два варианта:

- не брать текущий элемент;
- взять его и добавить к лучшему решению для позиции $i - 2$.

Получаем рекуррентную формулу:

$$dp[i] = \max\left(dp[i-1], dp[i-2] + a[i]\right).$$

Пример реализации:

```
dp = [0] * len(a)

dp[0] = a[0]
dp[1] = max(a[0], a[1])

for i in range(2, len(a)):

    dp[i] = max(dp[i-1],
                dp[i-2] + a[i])

print(dp[-1])
```

Таким образом, мы последовательно вычисляем оптимальный результат для всей последовательности.

2.6.2 Практические шаблоны

Пример

Поиск максимального элемента

```
arr = [4,7,2,9,5]

best = arr[0]

for x in arr:
    best = max(best, x)

print(best)
```

Пример

Баланс в бинарной строке

```
s = "010011"

balance = 0
count = 0

for c in s:

    if c == '1':
        balance += 1
    else:
        balance -= 1

    if balance == 0:
        count += 1

print(count)
```

Пример

Слияние двух отсортированных списков

```
a = [1,4,7]
b = [2,3,8]

i = 0
j = 0
res = []

while i < len(a) and j < len(b):

    if a[i] < b[j]:
        res.append(a[i])
        i += 1
    else:
        res.append(b[j])
        j += 1

res += a[i:]
res += b[j:]

print(res)
```

2.7 Алгоритмические конструкции

2.7.1 Разбор задач прошлых лет

2025

Задача

Рассмотрим следующую функцию.

```
def check(n):

    while n > 1:
        if n % 3 != 0:
            return False
        n //= 3

    return True
```

Определите асимптотическую сложность этого алгоритма.

Решение

На каждой итерации цикла значение переменной n делится на 3:

$$n \to \frac{n}{3}.$$

Таким образом, после k шагов получаем

$$\frac{n}{3^k}.$$

Цикл заканчивается, когда значение становится меньше либо равно 1.
Следовательно,

$$3^k \approx n.$$

Отсюда

$$k \approx \log_3 n.$$

Количество итераций цикла пропорционально логарифму от n.
Следовательно, асимптотическая сложность алгоритма равна

$$O(\log n).$$

2025

Задача

Число называется *интересным*, если его простые множители могут быть только из множества

$$\{3, 5, 11\}.$$

Другими словами, любое интересное число имеет вид

$$3^a \cdot 5^b \cdot 11^c.$$

Необходимо найти максимальное интересное число, не превосходящее 10000.

Решение

Любое интересное число имеет вид

$$3^a \cdot 5^b \cdot 11^c.$$

Поэтому можно перебрать все возможные значения степеней a, b и c, пока значение числа не превышает 10000.
Алгоритм:

1. перебрать степени числа 3;
2. перебрать степени числа 5;
3. перебрать степени числа 11;
4. вычислить произведение;
5. выбрать максимальное значение, не превышающее 10000.

Пример реализации:

```
limit = 10000
best = 0

a = 1
while a <= limit:

    b = a
    while b <= limit:

        c = b
        while c <= limit:

            best = max(best, c)
            c *= 11

        b *= 5

    a *= 3

print(best)
```

Таким образом перебираются все числа вида

$$3^a 5^b 11^c.$$

2025

Задача

Необходимо заполнить таблицу 4×4 числами $1, 2, 3, 4$ так, чтобы в каждой строке и в каждом столбце все числа были различны. Сколько различных таблиц можно получить?

Решение

Такая таблица называется **латинским квадратом** порядка 4. Латинский квадрат — это таблица, в которой каждое число встречается ровно один раз в каждой строке и в каждом столбце.
Найдём количество таких таблиц с помощью полного перебора. Будем последовательно заполнять клетки таблицы и на каждом шаге проверять, не нарушается ли условие задачи: в строке и в столбце не должно быть повторяющихся чисел.
Если число можно поставить, переходим к следующей клетке. Если нет — пробуем другое значение. Такой метод называется **поиском с возвратом (backtracking)**.
Пример реализации на Python:

```
n = 4
grid = [[0] * n for _ in range(n)]
count = 0

def valid(r, c, val):
    for i in range(n):
        if grid[r][i] == val:
            return False
        if grid[i][c] == val:
            return False
    return True

def backtrack(pos):
    global count

    if pos == n * n:
        count += 1
        return

    r = pos // n
    c = pos % n

    for val in range(1, n + 1):
        if valid(r, c, val):
            grid[r][c] = val
            backtrack(pos + 1)
            grid[r][c] = 0

backtrack(0)
```



```
print(count)
```

Программа перебирает все допустимые заполнения таблицы и подсчитывает их количество.

2.7.2 Практические шаблоны

Пример

Проверка простого числа

```
def is_prime(n):

    if n < 2:
        return False

    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False

    return True

print(is_prime(17))
```

Пример

Перебор степеней чисел

```
limit = 10000
best = 0

a = 1

while a <= limit:

    b = a

    while b <= limit:

        c = b

        while c <= limit:

            best = max(best, c)
            c *= 11
```

```
        b *= 5

    a *= 3

print(best)
```

Пример

Динамическое программирование

```
a = [3,7,4,6,5]

dp = [0] * len(a)

dp[0] = a[0]
dp[1] = max(a[0], a[1])

for i in range(2, len(a)):
    dp[i] = max(dp[i-1],
                dp[i-2] + a[i])

print(dp[-1])
```

Тренировочное тестирование

Этот тест предназначен для проверки готовности к выполнению конкурсного тестирования.

Рекомендуемое время выполнения — **60 минут**.

Ответом в каждой задаче является результат работы программы.

—

Задача 1

Дана строка

```
s = "abacbbdacabcbbadabccbabdabcbbac"
```

Необходимо определить, сколько различных символов встречается в строке **более трёх раз**.

В ответе укажите одно число.

—

Задача 2

Дан список чисел

```
data = [
3,5,3,7,5,3,7,7,7,8,8,8,8,
9,9,2,2,2,2,2,4,4,4,6,6,6,6
]
```

Необходимо определить число, которое встречается чаще всего.

Если таких чисел несколько, необходимо вывести **наибольшее из них**.

—

Задача 3

В файле содержатся числа

```
numbers = [
12,45,23,67,89,34,21,90,56,78,
11,22,33,44,55,66,77,88,99,100
]
```

Необходимо:

- найти среднее значение всех чисел;
- определить, сколько чисел **строго больше среднего**.

В ответе укажите это количество.

—

Задача 4

Дан неориентированный граф, заданный матрицей смежности.

```
graph = [
[0,1,1,0,0,0],
[1,0,1,1,0,0],
[1,1,0,0,1,0],
[0,1,0,0,1,1],
[0,0,1,1,0,1],
[0,0,0,1,1,0]
]
```

Необходимо определить:

- степень каждой вершины;
- номер вершины с максимальной степенью.

Если таких вершин несколько, выведите **наименьший номер**.

—

Задача 5

Дана последовательность чисел

```
a = [5,12,7,3,14,9,6,11,4,10]
```

Необходимо определить максимальную сумму подпоследовательности, в которой нельзя выбирать два соседних элемента.

—

Задача 6

Дана бинарная строка

```
s = "110010011101000110101001"
```

Известно, что количество нулей равно количеству единиц.

Необходимо разбить строку на максимальное количество подряд идущих подстрок, в каждой из которых количество нулей равно количеству единиц.

В ответе укажите это количество.

—

Задача 7

Рассмотрим следующий код.

```
n = 1

while n < 10**7:
    n *= 3

print(n)
```

Определите, какое число выведет программа.

—

Задача 8

Дана таблица

```
grid = [
[1,2,3,4],
[2,3,4,1],
[3,4,1,2],
[4,1,2,3]
]
```

Необходимо определить, является ли таблица латинским квадратом.

Напишите программу, которая выводит

```
True
```

или

```
False
```